REACH

Build Pack for Antigravity and Codex

Prompts, instructions, links, demo plan and the full build spec

ZERO ONE Hackathon, 12 Oct 2026

1. How to use this pack

- `BUILD_SPEC.md` is the file the AI agent reads. Put it in the root of an empty project folder.
- This PDF is for you: prompts to paste, links, what to prepare, and the demo plan. The full spec is also at the end of this PDF.
- Order: put BUILD_SPEC.md in the folder, open the folder in Antigravity, paste the Master Prompt, then use the Phase Prompts one by one.

2. Master prompt (paste this first in Antigravity)

```
You are building REACH, a hackathon project that must win.
Read BUILD_SPEC.md in the project root completely before you write any code.
Follow it exactly: the tech stack, pages, design system, simple-English copy,
gesture engine, WebSocket messages, and the acceptance checklist.

Rules:
1. Work in phases (section 14 of the spec). Do Phase 1 now, then stop.
2. After each phase run the app, open it in the browser, test it, take
   screenshots, fix bugs, then tell me exactly what I should check.
3. Never fake results. If the camera or model fails, show a clear message.
4. The UI must look premium: calm, spacious, large buttons, smooth motion.
   Use very simple English on every screen.
5. Everything must work offline on one Wi-Fi network. Bundle the MediaPipe
   model and WASM files locally.
Start Phase 1.
```

3. Phase prompts (paste one at a time, after you tested the last phase)

Phase 2:

```
next. Do Phase 2: the gesture engine (normalize, features, kNN, rest class,
state machine) with unit tests, plus the full /teach flow and saving the
profile. Make teaching one gesture take under 20 seconds.
```

Phase 3:

```
next. Do Phase 3: Express + WebSocket server, /live sending requests,
/caregiver receiving them, Acknowledge, and the "Help is coming" message
on the patient screen. Show the QR code and the phone URL on /live.
```

Phase 4:

```
next. Do Phase 4: phone polish. Vibration, chime, text-to-speech, urgent
red flash, escalation after 30 seconds, quick replies, history and CSV.
Test the caregiver page at 360px width.
```

Phase 5:

```
next. Do Phase 5: Gesture Coach, comfort score, movement map, the /results
page with accuracy test and 60-second false-alarm test, and /settings.
```

Phase 6:

```
next. Do Phase 6: polish everything, write the README with screenshots,
add the demo reset button and friendly empty and error states, then go
through the acceptance checklist and fix every item that fails.
```

If something breaks:

```
Something is wrong: [describe it]. Reproduce it in the browser first,
find the root cause, fix it, and add a test so it cannot come back.
```

Look-and-feel fix prompt (use if the UI looks plain):

```
The UI looks too plain. Redo the visual design to match the Design system
in BUILD_SPEC.md section 4: more white space, bigger type, soft shadows,
rounded cards, the teal accent, smooth Framer Motion transitions, and a
beautiful camera frame with a glowing hand skeleton and hold ring.
Show me before and after screenshots.
```

4. Antigravity or Codex? Simple plan

- Antigravity = main builder. It is an agent-first IDE with a browser it can drive, so it can build the screens and check them by looking at them. Let it do Phases 1-6.
- Codex = second pair of hands. Give it jobs that do not touch the UI files at the same time: (1) write extra unit tests for `src/engine/`, (2) review the state machine for false-alarm bugs, (3) write the README and the CSV export, (4) fix bugs from a bug list.
- Do not let both tools edit the same files at the same time. Use git: Antigravity works on `main`, Codex works on a branch like `codex/tests`, and you merge when each is done.
- Codex prompt example:

```
Read BUILD_SPEC.md sections 6 and 9. Review src/engine and server/.
Write unit tests for normalization, kNN confidence, the rest class and every
state in the state machine. Try to make it send a false request. Report bugs
first, then fix them on a new branch. Do not change the UI files.
```

5. What you need to prepare (checklist)

- Laptop with a working webcam, Chrome (latest), Node.js 20 or newer, Git. A GitHub repo (public, with README) is a hackathon rule.
- A phone (Android is best because its browser can vibrate) on the same Wi-Fi as the laptop.
- IMPORTANT: college or hostel Wi-Fi often blocks devices from talking to each other ("client isolation"). Test this today. Backup plan: turn on your phone's hotspot and connect the laptop to it. Then the phone and laptop are on one private network.
- Windows firewall: the first time Node runs, allow it on "Private networks" or the phone cannot connect.
- The camera works on `localhost`, but not on `http://192.168...`. So run the patient screen on the laptop that runs the server. The caregiver screen (no camera needed) can open from any device.
- iPhone note: Safari cannot vibrate from a web page. It will still play sound and flash the screen. Use an Android phone for the buzz moment if you can.
- Good light on your hand, plain background. Test the exact table and light you will have at the hackathon.
- Optional: install the free ntfy app on the phone for lock-screen alerts (extra, needs internet).
- Practise with 2-3 different people's hands so the judge demo does not surprise you.

6. Extras that can make it win (already in the spec, in this order of value)

1. Gesture Coach: tells the user when two gestures look too alike and which one feels tiring. Nobody else will have this.
2. Judge teaches live in under 20 seconds, with a visible "Time to teach" timer.
3. Results page with REAL numbers: accuracy, false alarms in 60 seconds, delivery speed. Put these on your slides.
4. Privacy badge: "Your video stays on this laptop. Only hand points are used." Judges like this a lot in health.
5. Escalation: if no one acknowledges in 30 seconds, it gets louder and moves to the top. This is how real nurse calls work.
6. Two-way talk: the caregiver sends "Coming now" or "2 minutes" and the patient hears it.
7. Fatigue-aware softening (stretch).
8. Works offline. Say it on stage.
9. Keyboard-simulated gestures as a safety net if the camera fails on stage.

7. Winning tips for the pitch

- Rules say 5-minute pitch, 2-minute Q&A, 7 slides. Spend most of your time on the live demo, not slides.
- Open with a person, not a technology: "Imagine you cannot speak and you cannot press the button. How do you ask for water?"
- Demo script (about 2 minutes): judge teaches a gesture (20 s) -> uses it -> phone buzzes -> caregiver taps Acknowledge -> patient screen says "Help is coming" -> show random waving causing no request -> open /results.
- Likely questions and honest answers:
  - "How is this different from Teachable Machine?" It is a full care workflow: priorities, two-way acknowledgement, false-alarm control and a coach, built for people who cannot use standard gestures.
  - "Is it medical grade?" No. It is a prototype and the About card says what testing it would need.
  - "What if the person cannot move their hand at all?" Start with hand and wrist. The same engine can use other points later (head, eyes) - that is future work, not a claim.
  - "Privacy?" No video is saved or sent. Only hand points.
  - "Why not a normal call button?" Some people cannot press one: stroke, paralysis, weak grip, after surgery, hands in pain, ICU.
- Do not quote statistics unless you checked the source yourself and can show it.
- Competition to mention (check each before you say it): eye-gaze devices (for example Tobii Dynavox) need costly hardware; normal call buttons and pillow switches need a hand press; fixed-gesture systems make the person learn the movements; Teachable Machine and Project Gameface show the idea is real but are not a care workflow.
- Do the LinkedIn post and the repo README before Monday. They are required.

8. Timeline (today is Sat 10 Oct, hackathon is Mon 12 Oct, 9 AM to 4 PM)

- Today: Phases 1-3. End the day with: teach a gesture, send it, see it on a second tab.
- Tomorrow (Sun): Phases 4-6. Test on your phone, test on the hotspot, record a backup demo video, finish README, make the 7 slides, LinkedIn post.
- Monday morning: do not add features. Pull, run, test the full demo three times, charge everything.
- Rule: if a feature is not working by Sunday night, cut it. A smaller thing that works every time beats a bigger thing that fails on stage.

9. Website and design links

Design inspiration (open these and show them to the agent if you want a certain look):

- Linear (clean, modern, calm): https://linear.app
- Stripe (premium spacing and gradients): https://stripe.com
- Apple Accessibility (tone for accessible products): https://www.apple.com/accessibility/
- Awwwards (award-winning sites): https://www.awwwards.com
- Dribbble healthcare dashboards: https://dribbble.com/tags/healthcare-dashboard
- Mobbin (real mobile app screens): https://mobbin.com
- Radix Colors (nice color scales): https://www.radix-ui.com/colors
- Teachable Machine (the "teach it live" feel): https://teachablemachine.withgoogle.com
- Google Project Gameface (camera-based accessibility, open source): https://github.com/google/project-gameface

Build tools:

- MediaPipe Hand Landmarker (web guide): https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/web_js

- MediaPipe live demo: https://mediapipe-studio.webapps.google.com/studio/demo/hand_landmarker
- MediaPipe package: https://www.npmjs.com/package/@mediapipe/tasks-vision
- Hand model file: https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task
- Vite: https://vite.dev
- React: https://react.dev
- Tailwind CSS: https://tailwindcss.com
- shadcn/ui (nice ready components): https://ui.shadcn.com
- Motion (Framer Motion): https://motion.dev
- Lucide icons: https://lucide.dev
- Google Fonts: https://fonts.google.com
- ws (WebSocket for Node): https://github.com/websockets/ws
- qrcode package: https://www.npmjs.com/package/qrcode
- ntfy (free phone push): https://ntfy.sh
- Vibration API: https://developer.mozilla.org/en-US/docs/Web/API/Vibration_API
- Speech synthesis: https://developer.mozilla.org/en-US/docs/Web/API/SpeechSynthesis
- Screen Wake Lock: https://developer.mozilla.org/en-US/docs/Web/API/Screen_Wake_Lock_API
- NoSleep.js (keep phone awake): https://github.com/richtr/NoSleep.js
- WCAG quick reference: https://www.w3.org/WAI/WCAG22/quickref/

I have not opened each link today, so click through them once to confirm they still load.

Full build spec (same as BUILD_SPEC.md)

0. How to work

- Read this whole file before writing code. Follow it closely.
- Build in phases (section 14). After each phase: run the app, test it in the browser, fix bugs, then tell me what to check. Stop and wait for "next".
- Never fake results. If the camera, model or network fails, show a clear, friendly message.
- Everything must work with NO internet on one Wi-Fi network (the AI model files are bundled locally).
- One command to start: `npm install` then `npm run dev`. Prints the laptop URL and the phone URL.
- Keep the code clean and commented. Add a README with setup steps, screenshots and a short architecture diagram.

1. What REACH is

REACH is a camera-based communication tool for people who cannot speak or press a normal call button. It learns the movement a person CAN make and turns it into a request for a caregiver (Water, Pain, Bathroom, Move me, Emergency). The person teaches it. It does not force the person to learn gestures.

One line: "REACH learns the movement the person can make, instead of asking the person to learn the movement the system expects."

Hackathon rules to respect: at least 5 gestures controlling a real app, clear on-screen feedback, ambiguous gestures must be ignored (never trigger by mistake).

2. Tech stack (use this unless something is impossible)

- Frontend: Vite + React + TypeScript + Tailwind CSS + Framer Motion + lucide-react icons. React Router for pages.
- Hand tracking: `@mediapipe/tasks-vision` HandLandmarker, running in the browser. Copy the WASM files and `hand_landmarker.task` into `public/mediapipe/` so it works offline. Use GPU delegate, fall back to CPU.
- Backend: Node.js + Express + `ws` (WebSocket). Saves data to `data/*.json`. Serves the built frontend too, so phones can open it from the laptop IP.
- QR code: `qrcode` npm package.
- No database server, no login, no cloud. No video is ever uploaded or saved. Only 21 hand points per frame are used.
- (If I ask for Python instead, FastAPI + WebSocket works too. Keep the same message format from section 9.)

3. Pages

Route	Who	Purpose
`/`	everyone	Home. Three big cards: "I need help" (patient), "I am a caregiver", "Judge demo".
`/teach`	patient / judge	Teach REACH movements (section 5).
`/live`	patient	Live screen. Camera, status, request chips, "Help is coming" message.
`/caregiver`	caregiver (laptop or phone)	Request list, Acknowledge, history. Works well on a phone.
`/results`	judges	Numbers: speed, accuracy, false alarms, confusion table (section 11).
`/settings`	anyone	Sliders for hold time and strictness, sound, language, theme.

Navigation: simple top bar with the REACH logo, page links, and a small "Private: video never leaves this device" badge with a lock icon.

4. Design system (must look premium, calm and modern - not like a boring hospital app)

Mood: calm, warm, trustworthy, a little futuristic. Think Apple Health meets Linear. Lots of space. Big touch targets.

- Colors (light): background `#F8F7F4`, cards `#FFFFFF`, ink `#0F172A`, muted text `#64748B`, primary teal `#0E9F9A` (hover `#0B7A75`), soft teal tint `#E6F6F5`.
- Priority colors: Normal = green `#22A06B`, Important = amber `#F5A524`, Urgent = red `#E5484D`. Never use color alone - always add an icon and a word.
- Dark theme for night shift: background `#0B1220`, cards `#111A2E`, same accents. Toggle in settings and remember it.
- Fonts (Google Fonts, but also bundle locally for offline): headings "Plus Jakarta Sans" 700, body "Inter" 400/500. Body text at least 18px. Patient screens at least 22px.
- Shapes: cards radius 24px, buttons radius 16px, buttons at least 64px tall on patient and phone screens. Soft shadows. Subtle glass effect only on the camera overlay.
- Motion (Framer Motion): gentle fade/slide on page change, spring pop when a request arrives, smooth progress ring. Respect `prefers-reduced-motion`.
- Camera view: mirrored video in a large rounded frame. Draw the hand skeleton as a thin glowing teal line with dots. Show a circular "hold ring" that fills while a gesture is being held. Show the matched request name big under the ring.
- Hero touch on Home: soft animated gradient blob behind a looping hand-skeleton animation (SVG).
- Accessibility: WCAG AA contrast, visible keyboard focus, `aria-live` for new requests, large text option in settings.
- Empty states are friendly and have one clear next action.

5. Teach flow (`/teach`) - the main demo moment

Make it feel effortless. A whole gesture should take under 20 seconds to teach. Show a visible timer called "Time to teach" (the judges will love it).

Steps (show a 4-dot progress bar):

1. "Who is this for?" Name + bed/room (e.g. "Ravi, Bed 4"). One button: Continue.
2. "Pick what you want to say". Grid of big cards: Water, Pain, Bathroom, Move me, Emergency, Food, + "Make your own" (name + pick priority). The user picks at least 3, up to 8.
3. "Show your movement". For each chosen request:

- Ask: "Do a movement you can do easily. A hand shape or a small wrist move is fine."

- Choose type with two big buttons: "Hold a hand shape" or "Do a small move" (wave, tilt, tap, circle).

- Countdown 3-2-1, then record. Hand shape: capture while the person holds it for about 1.5 seconds, 3 rounds, with "relax" between rounds, about 8-10 samples per round. Small move: record 5 repeats of about 1.5 seconds each.

- Live preview while recording, with a "Got it 1 of 3" counter.

4. "Rest hand" step: "Now just relax or move your hand around normally for 5 seconds." Save these as "not a request" samples. This is what stops false alarms.
5. "Try it now". Free test screen. Every gesture shows how sure REACH is. Green tick when it matches. Buttons: "Teach again" per gesture, "Done".

Gesture Coach (important, makes us stand out):

- After teaching, compare all gestures. If two are too similar, show: "Water and Pain look alike. Try a different movement for one of them." with a one-click re-teach.

- Show a comfort score per gesture: "Easy for you" (samples very consistent) or "Might be tiring" (samples vary a lot). Offer to re-teach a simpler one.
- Show a simple "movement map" (2D scatter of all samples, one color per request, made with PCA or just 2 dimensions) so judges can SEE the gestures are separate. Keep it small and pretty.

Saving: after teaching, auto-save the profile to the server (`PUT /api/profile/:id`) and to localStorage. Reopening the app must not require re-teaching. Show a "Saved on this laptop" check.

6. Gesture engine (put it in `src/engine/`, keep it independent from React, unit-test it)

Step A - landmarks: one hand, 21 points (x, y, z) per frame. Run at 15 FPS or more. Show FPS in the debug panel.

Step B - normalize so it works at any distance, position or hand size:

- Subtract the wrist point. Divide by the distance wrist to middle-finger knuckle.
- Rotate so wrist to middle knuckle points up (setting: "ignore hand tilt", default ON).
- If it is a left hand, mirror x so left and right hands share one profile.

Step C - features:

- Pose gestures: the 63 normalized numbers plus 5 fingertip-to-wrist distances plus 5 finger-bend angles.
- Move gestures: keep the last 20 frames, take the palm-center path, subtract the first point, scale by hand size, resample to 16 points, flatten. Also add average speed. Start a move recording when the hand starts moving and stop when it rests.

Step D - classifier (instant, works with few samples, no heavy training):

- k-nearest-neighbors (k=3) on stored samples. Training = just saving samples, so it is instant.
- For each request, d = average of its 3 nearest distances. Let s = typical spread of that request's own samples (with a small minimum).
- confidence = clamp(1 - d / (3 * s), 0, 1). margin = best confidence minus second-best confidence.
- If the closest sample is a "rest" sample, the answer is "nothing".
- You may improve this, but you must keep: a rest class, a confidence number, a margin check, and rejection of unsure results.

Step E - decision state machine (this prevents false alarms):

```
NO_HAND -> LOOKING -> CANDIDATE -> HOLDING -> SENT -> WAITING -> ACKED -> COOLDOWN
```

- CANDIDATE: same label for at least 5 frames in a row with confidence >= minConfidence (default 0.75) and margin >= minMargin (default 0.15).
- HOLDING: keep the same label for holdMs (default 1200 ms). The hold ring fills. If the label changes or confidence drops, the ring empties and nothing is sent.
- Emergency (Urgent) uses two-stage confirm: hold the Emergency gesture, then show a confirm message "Hold again to confirm" for 4 seconds. Holding it again sends. Doing nothing cancels. Settings can turn this off.
- SENT: send the request. Then COOLDOWN for cooldownMs (default 5000) so one gesture does not send ten requests.
- Unsure result: show "Not sure. Try again." in calm yellow. Never send.
- Stretch: a learned "Cancel" gesture that cancels the last request.
- Stretch (fatigue-aware): if the same gesture fails 3 times in a row with weaker and weaker confidence, show "Take your time" and lower minConfidence a little for the next 60 seconds. Show a small note when this is active.

7. Live screen (`/live`)

- Large mirrored camera view with skeleton + hold ring (section 4).

- Top strip: connection dot ("Connected to caregiver screen" / "Offline - saving requests"), battery-style FPS indicator.
- Below camera: chips for each taught request (icon + name + priority color). The chip lights up when its gesture is seen.
- When a request is sent: full-width friendly card "We told your caregiver: Water". Then waiting animation.
- When acknowledged: green card with caregiver name: "Priya got your message. Help is coming." Also speak it with text-to-speech (setting).
- Show "Notify me on" row with a QR code: "Scan to open the caregiver screen on a phone" + the URL as text (from `GET /api/info`). Also a button "Open caregiver screen in a new tab" (for one-laptop demos).
- Keyboard shortcuts 1-8 simulate each gesture (demo safety net). Hide this behind Settings -> "Demo helpers".
- If the server is unreachable, queue requests in memory and send when it comes back.

8. Caregiver screen (`/caregiver`, designed phone-first, looks great on a laptop too)

- Header: caregiver name (ask once, save), connection status, sound on/off, "Tap to enable alerts" button (browsers need a tap before sound and vibration).
- List of live requests, urgent first, then oldest first. Each card: big icon, request word, patient + bed, how long ago (live ticking), priority badge, "How sure: 92%", and two buttons: ACKNOWLEDGE (primary, huge) and "Done" (after acknowledged).
- New request effects: spring-in animation, short chime (Web Audio), vibration pattern on phones (`navigator.vibrate`: normal [200], important [200,100,200], urgent [500,150,500,150,500]), and text-to-speech "Bed 4, Ravi needs water" (setting).
- Urgent: whole screen edge flashes red until acknowledged.
- Quick replies after acknowledging: "Coming now", "2 minutes", "Will bring it". Sent to the patient screen.
- Escalation: if a request is not acknowledged in 30 seconds (setting), mark it "Still waiting", make it louder and move it to the top.
- History tab: all requests with time, who acknowledged, and how fast. Button to download CSV.
- Phone note: iPhones do not support vibration in the browser. On iPhone use sound + screen flash. Keep the screen awake using the Wake Lock API when allowed, otherwise the NoSleep.js trick.

9. Server and message format

HTTP:

- `GET /api/info` returns `{ lanUrl, caregiverUrl, version }` (find the real Wi-Fi IP, not 127.0.0.1)
- `GET /api/profile/:id`, `PUT /api/profile/:id`
- `GET /api/requests`, `GET /api/requests.csv`
- `POST /api/reset-demo` clears requests, keeps profiles

WebSocket at `/ws`. All messages are JSON with a `type`.

- Client to server: `{type:"hello", role:"patient"|"caregiver", name}`
- Patient to server: `{type:"request", request:{id, patientId, patientName, bed, label, priority:"normal"|"important"|"urgent", confidence, triggeredAt}}`
- Server to caregivers: `{type:"request:new", request:{...,receivedAt, status:"new"}}`
- Caregiver to server: `{type:"ack", id, caregiverName, quickReply?}`, `{type:"resolve", id}`
- Server to patient: `{type:"request:acked", id, caregiverName, quickReply?}`
- Server to patient after a caregiver screen shows it: `{type:"request:delivered", id, ms}` (used for the speed number in section 11)

- Server to all: `{type:"request:escalated", id}`, `{type:"presence", caregivers:n, patients:n}`
- Auto-reconnect with backoff on all clients. The server keeps every request in memory and in `data/requests.json`. On a new connection, send the current open requests.

Profile shape:

```
{id, name, bed, gestures:[{id,label,priority,kind:"pose"|"move",samples:number[][],createdAt}],
restSamples:number[][],
settings:{holdMs,minConfidence,minMargin,cooldownMs,ignoreTilt,twoStepUrgent,speak,largeText}}
```

10. Simple English rules (very important)

- Write at about a Grade 5 reading level. Short sentences. Everyday words. No medical jargon, no tech words on patient or caregiver screens.
- Say "How sure" instead of "confidence". Say "movement" instead of "gesture" on patient screens. Say "Teach" instead of "train".
- Buttons start with a verb: "Teach a movement", "Try it now", "Acknowledge".
- Friendly errors that say what to do: "I can't see your hand. Move it into the picture." "It is too dark. Turn on a light."
- Technical words are allowed only on `/results` and the debug panel.
- Copy examples: "Teach REACH a movement", "Pick what you want to say", "Do it 3 times. Relax in between.", "Great! REACH learned Water.", "Not sure. Try again.", "We told your caregiver.", "Priya got your message. Help is coming.", "Your video stays on this laptop."

11. Results page (`/results`) - this wins the "implementation" score

- Live numbers: FPS, time from hand-ready to request sent, time to reach the caregiver screen (median of last 20), requests today.
- "Run accuracy test" button: asks the person to do each gesture 5 times, then shows a confusion table (rows = asked, columns = recognized) and an accuracy percentage.
- "Run false-alarm test" button: 60-second timer. The person talks, scratches, waves around. Shows how many requests were wrongly sent (goal is zero).
- Show a small chart of confidence over time.
- Everything can be exported as JSON for the README and slides. Show only real measured numbers - never invented ones.

12. Extra features to add after the core works (in this order)

1. Gesture Coach + comfort score + movement map (section 5).
2. Escalation timer + quick replies (section 8).
3. Fatigue-aware softening (section 6).
4. Multi-bed view: caregiver screen can show 3 demo patients (Bed 2, 4, 7) with a "Demo patients" button that sends sample requests. Good for the market story.
5. ntfy.sh push (optional): if a topic name is set in Settings, also send an HTTP POST to `https://ntfy.sh/<topic>` so a phone gets a lock-screen notification even when the browser is closed. Needs internet. The app must work fully without it.
6. Sound themes and "quiet mode" (visual only).

13. Quality bar / acceptance checklist

[ ] Fresh laptop: `npm install && npm run dev` works, camera opens on `http://localhost`.

[ ] Works with Wi-Fi turned off (laptop only) and on a shared hotspot with a phone.

[ ] Teach 5 different requests in under 3 minutes. Each taught request is recognized at least 9 out of 10 tries in normal light.

[ ] Rest hand or random waving for 60 seconds sends zero requests.

- [ ] Two similar gestures trigger the Gesture Coach warning.
- [ ] A request appears on the caregiver screen in under 1 second and the phone buzzes (Android).
- [ ] Acknowledge on the caregiver screen shows "Help is coming" on the patient screen.
- [ ] Refreshing the page keeps the taught profile.
- [ ] No console errors. No layout break at 360px (phone), 768px, 1440px.
- [ ] Lighthouse accessibility score 90 or more on `/live` and `/caregiver`.
- [ ] README has: what it does, setup, how to demo, screenshots, architecture, limits, privacy.

14. Phases

1. Scaffold + design system + Home + camera + live hand skeleton + FPS.
2. Engine (normalize, features, kNN, state machine) with unit tests + `/teach` flow + saving.
3. Server + WebSocket + `/live` sending requests + `/caregiver` receiving, Acknowledge, "Help is coming".
4. Phone polish: QR code, vibration, sound, speech, escalation, history, CSV.
5. Gesture Coach, results page, accuracy and false-alarm tests, settings.
6. Final polish, README, demo reset button, empty and error states, extra features from section 12.

15. Honest limits (show on an About card and in the README)

REACH is a hackathon prototype. It is not a clinically tested medical device. Real use would need testing with real patients, safety checks, privacy review and approvals.